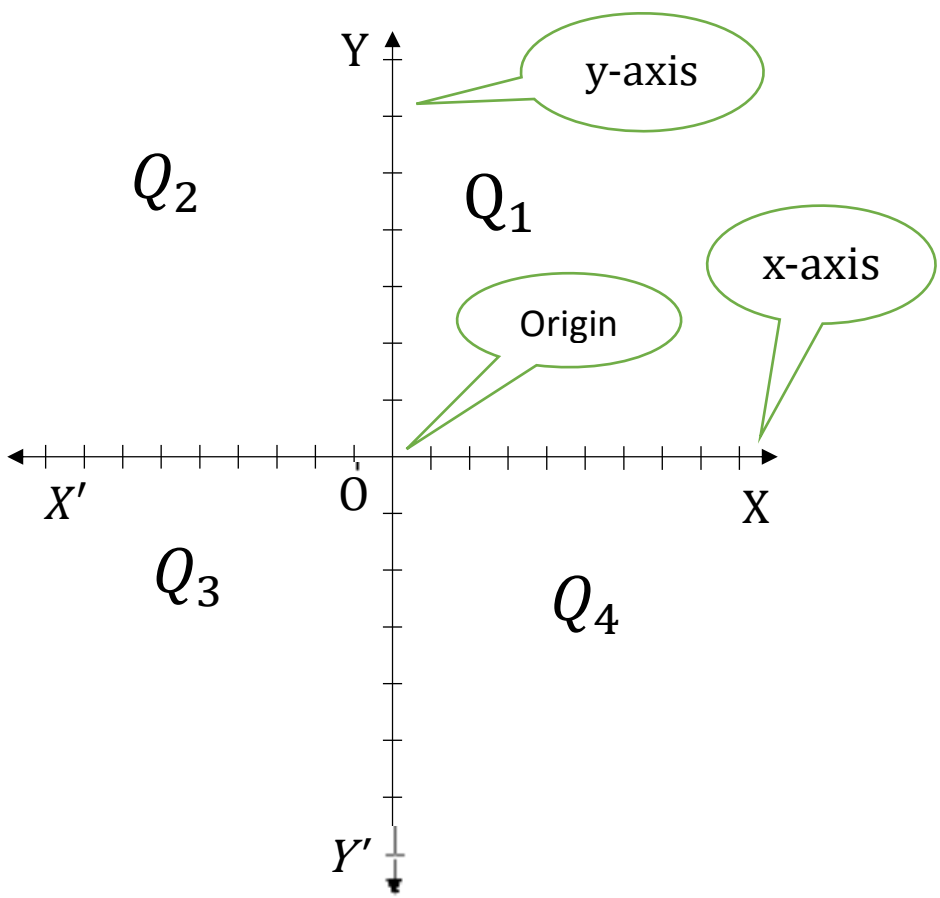
Topics:

- **Coordinate Systems**
- **Transformations**
- **Python Code: Applications in IT and Robotics**

Cartesian Coordinate System 2D

- Draw in a plane two mutually perpendicular number lines $x'x$ and $y'y$, horizontal and vertical called x-axis and y-axis respectively.
- The point of intersection is called "Origin" and is denoted by "O".
- The x-axis and y-axis have divided the plane into four parts, that are called "Quadrants". They are named Q_1, Q_2, Q_3, and Q_4.



Cartesian (Rectangular) Coordinate System 2D

In Cartesian Coordinates,

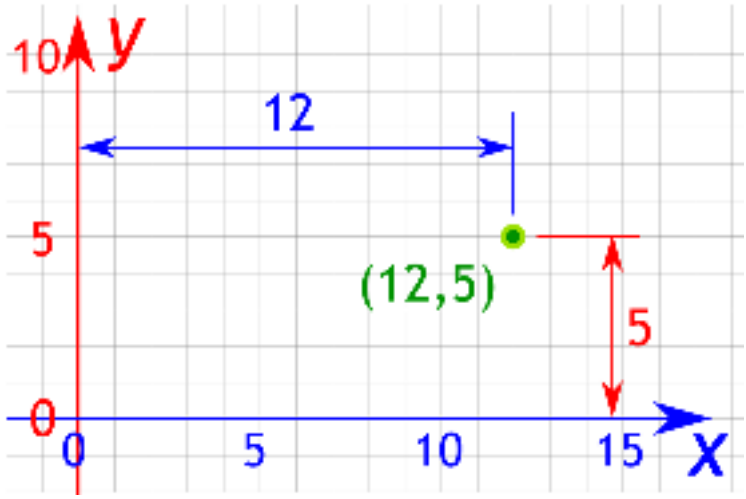
we mark a point by

how far along

and

how far up

it is



Polar Coordinate System 2D

In Polar Coordinates,

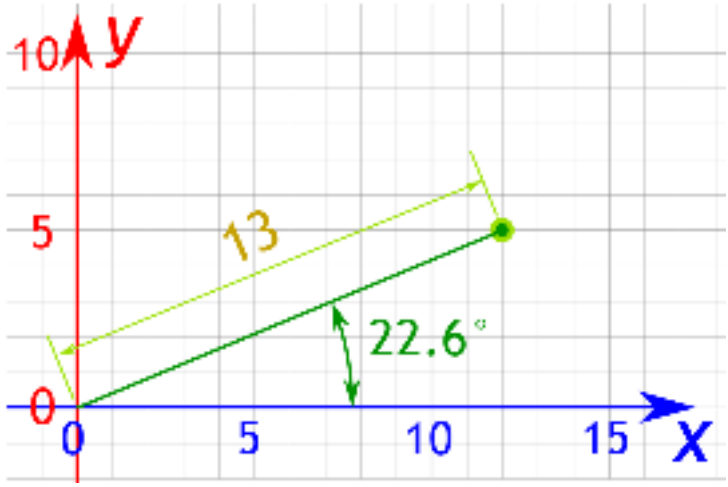
we mark a point by

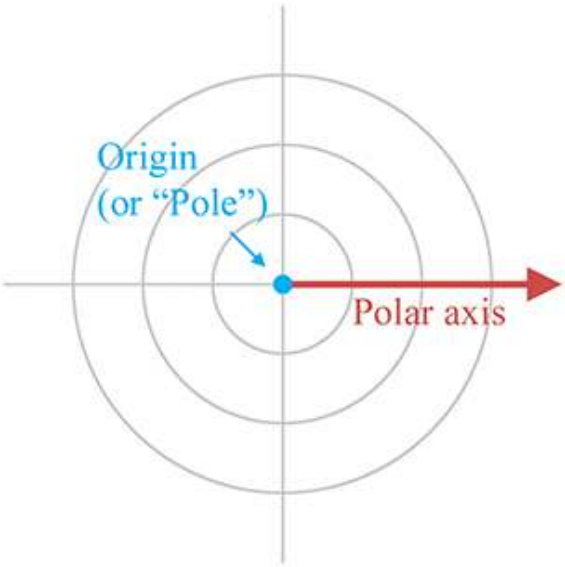
how far away

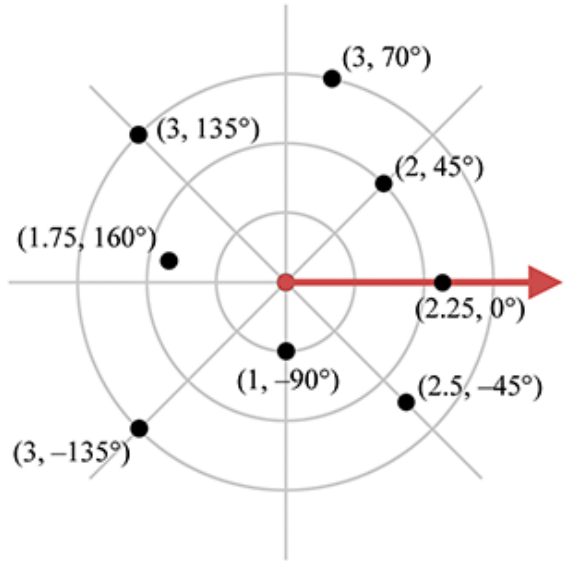
and

what angle

it is

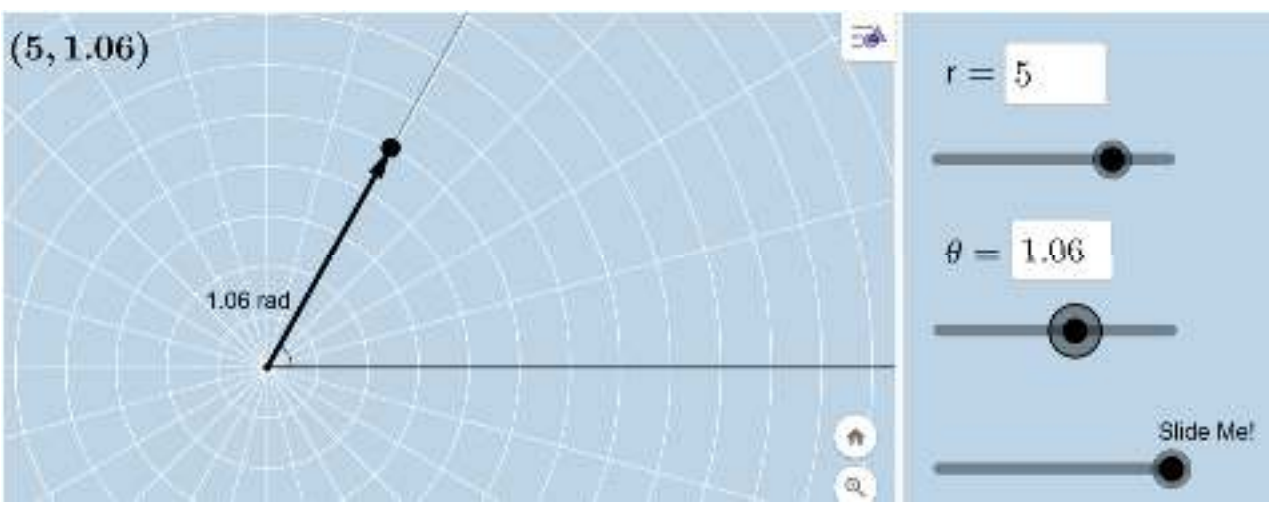






Visual explanation of Polar Coordinate

https://www.geogebra.org/classic/guCcvwFP



Conversion Formulas

From Cartesian to Polar	From Polar to Cartesian
• $r = \sqrt{x^2 + y^2}$	• $x = r \times \cos(\theta)$
• $\theta = \tan^{-1}(y / x)$	• $y = r \times \sin(\theta)$

Precaution …

when converting from Cartesian to Polar coordinates ...

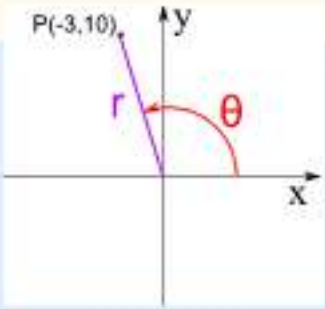
the calculator can give the **wrong value of $\tan^{-1}$**

It all depends what Quadrant the point is in!

Use following to fix things:

- Quadrant I: Use the calculator value
- Quadrant II: Add 180°
- Quadrant III: Add 180°
- Quadrant IV: Add 360°

Example: P = (−3, 10)



P is in **Quadrant II**

- $r = \sqrt{((-3)^2 + 10^2)}$
  $r = \sqrt{109} =$ **10.4** to 1 decimal place
- $\theta = \tan^{-1}(10/-3)$
  $\theta = \tan^{-1}(-3.33...)$

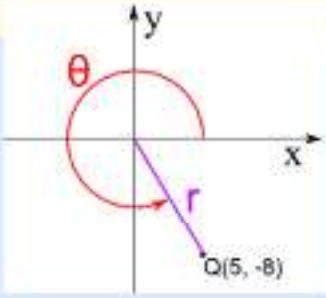
The calculator value for $\tan^{-1}(-3.33...)$ is −73.3°

The rule for Quadrant II is: **Add 180° to the calculator value**

$\theta = -73.3° + 180° = 106.7°$

So the Polar Coordinates for the point (−3, 10) are **(10.4, 106.7°)**

Example: Q = (5, −8)



Q is in **Quadrant IV**

- $r = \sqrt{(5^2 + (-8)^2)}$
  $r = \sqrt{89} =$ **9.4** to 1 decimal place
- $\theta = \tan^{-1}(-8/5)$
  $\theta = \tan^{-1}(-1.6)$

The calculator value for $\tan^{-1}(-1.6)$ is −58.0°

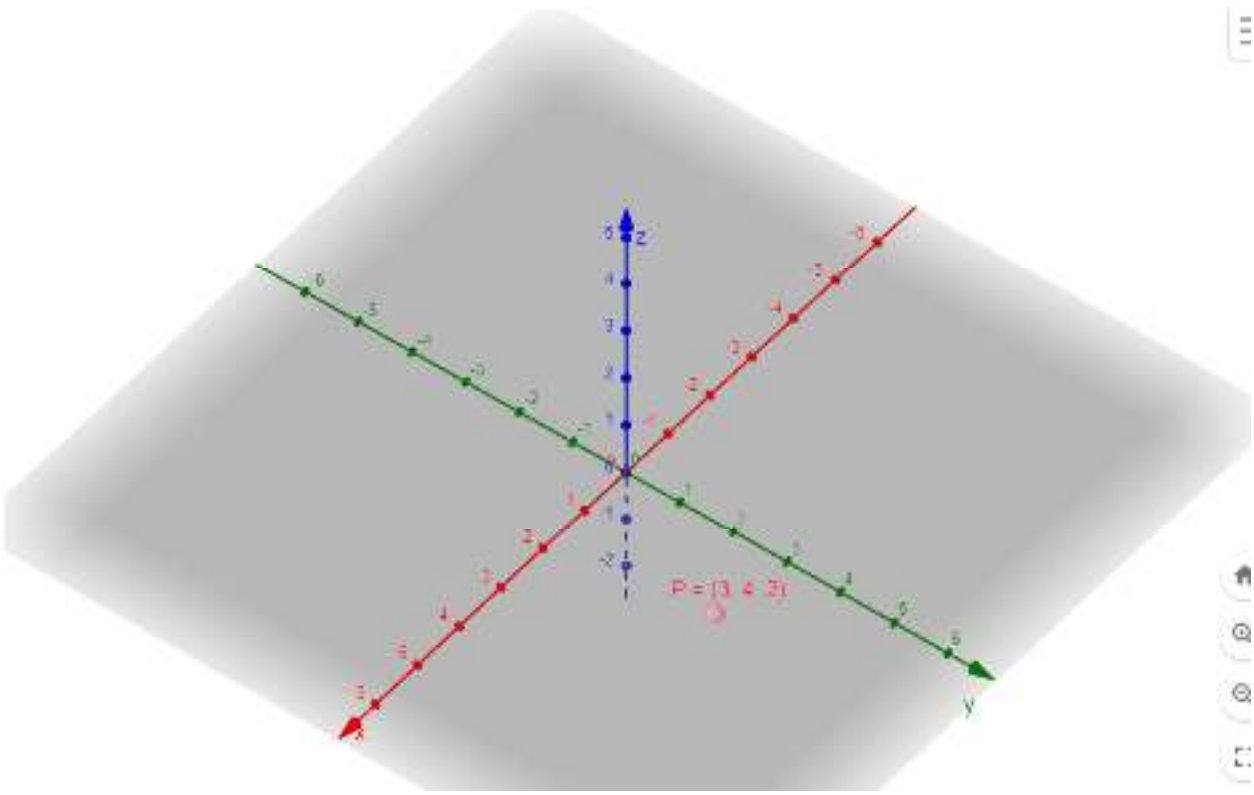
The rule for Quadrant IV is: **Add 360° to the calculator value**

$\theta = -58.0° + 360° = 302.0°$

So the Polar Coordinates for the point (5, −8) are **(9.4, 302.0°)**

Cartesian Coordinate System in 3D

- Definition and basics:
  - Axes $(x,\ y,\ z)$.
  - Representing points as $P(x, y, z)$
- Visualization in 3D:
  - planes (xy-plane, yz-plane, xz-plane).



Polar Coordinate System in 3D

As you probably have already guessed, 3D polar coordinates have *three* values. But is the third coordinate another linear distance (like r) or is it another angle (like θ)?

If we add a linear distance, we have *cylindrical coordinates*.

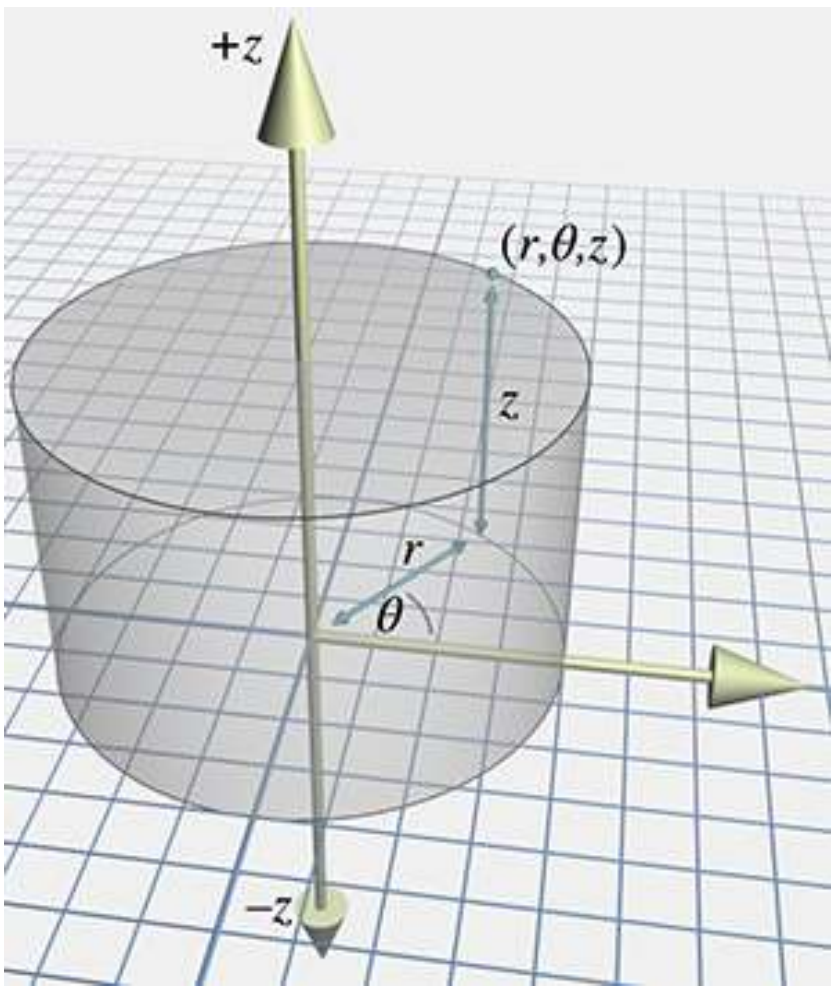
If we add another angle instead, we have *spherical coordinates*.

Cylindrical Coordinate System

The cylindrical coordinate system extends the polar coordinate system into three dimensions by adding a height component $\boldsymbol{z}$. It is particularly useful for modeling objects with cylindrical or rotational symmetry, such as pipes, cylinders, and spirals.

A point in the cylindrical system is described by $P(\boldsymbol{r}, \boldsymbol{\theta}, \boldsymbol{z})$.

- $\boldsymbol{r}$: The radial distance from the origin to the **projection** of the point on the $xy - Plane$
- $\boldsymbol{\theta}$: The angle between the positive $x - axis$ and the line connecting the origin to the **projection**.
- $\boldsymbol{z}$: The height of the point above or below the xy-plane, like Cartesian system.



Conversion Formulas

From Cylindrical to Cartesian	From Cartesian to Cylindrical
• $x = r\cos(\theta)$ • $y = r\sin(\theta)$ • $z = z$	• $r = \sqrt{x^2 + y^2}$ • $\theta = \tan^{-1}\left(\frac{y}{x}\right)$ • $z = z$

Spherical Coordinate System

The spherical coordinate system represents points in three-dimensional space using three values:

radial distance, polar angle, and azimuthal angle.

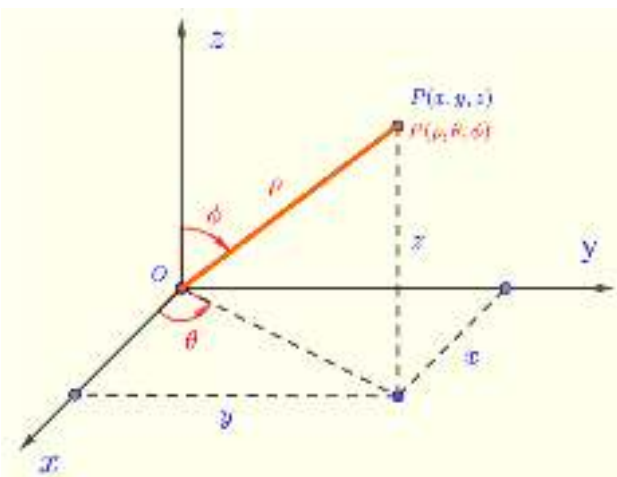
A point in the spherical system is described by $P(\rho, \theta, \phi)$

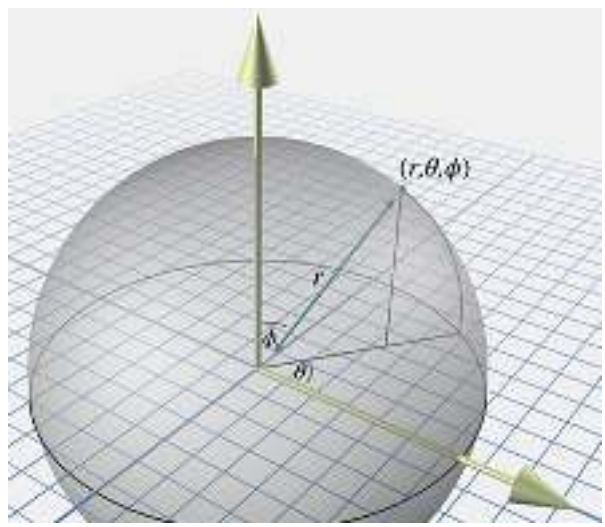
It is ideal for objects or problems involving spherical symmetry, such as planets, stars, or electromagnetic fields.

$\boldsymbol{\rho}$**:** The radial distance from the origin to the point (always non-negative).

$\boldsymbol{\theta}$: The angle, measured in the xy-plane from the positive $x-axis$ (similar to the angle in polar coordinates).

$\boldsymbol{\phi}$: The polar angle, measured from the positive z-axis down to the point.





Visual Representation

Cylindrical Coordinate System

https://www.geogebra.org/classic/X3j28ZkC

Spherical Coordinate System

https://www.geogebra.org/classic/P2RQ7SRz

Conversion Formulas

From Spherical to Cartesian	From Cartesian to Spherical
• $x = \rho \sin(\phi) \cos(\theta)$	• $\rho = \sqrt{x^2 + y^2 + z^2}$
• $y = \rho \sin(\phi) \sin(\theta)$	• $\theta = \tan^{-1}\left(\frac{y}{x}\right)$
• $z = \rho \cos(\phi)$	• $\phi = \cos^{-1}\left(\frac{z}{\rho}\right)$

▶ **Example 1**

(a) Find the rectangular coordinates of the point with cylindrical coordinates

$$(r, \theta, z) = (4, \pi/3, -3)$$

(b) Find the rectangular coordinates of the point with spherical coordinates

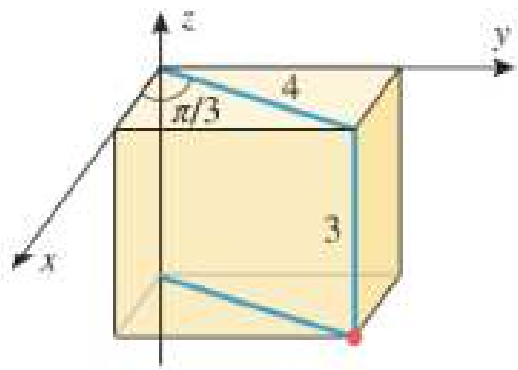
$$(\rho, \theta, \phi) = (4, \pi/3, \pi/4)$$

Solution (a)

$$x = r\cos\theta = 4\cos\frac{\pi}{3} = 2,$$

$$y = r\sin\theta = 4\sin\frac{\pi}{3} = 2\sqrt{3},$$

$$z = -3$$



cylindrical: $(4, \pi/3, -3)$
rectangular: $(2, 2\sqrt{3}, -3)$

(b) Find the rectangular coordinates of the point with spherical coordinates

$$(\rho, \theta, \phi) = (4, \pi/3, \pi/4)$$

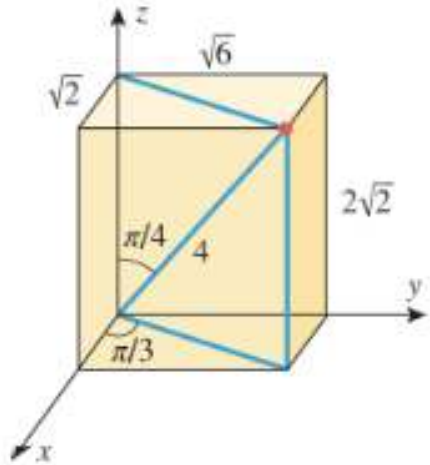
$$x = \rho \sin\phi \cos\theta = 4 \sin\frac{\pi}{4} \cos\frac{\pi}{3} = \sqrt{2}$$

$$y = \rho \sin\phi \sin\theta = 4 \sin\frac{\pi}{4} \sin\frac{\pi}{3} = \sqrt{6}$$

$$z = \rho \cos\phi = 4 \cos\frac{\pi}{4} = 2\sqrt{2}$$

So for, $(\rho,\ \theta,\ \phi) = \left(4, \frac{\pi}{3}, \frac{\pi}{4}\right)$

$$(x, y, z) = \left(\sqrt{2},\ \sqrt{6},\ 2\sqrt{2}\right)$$



▶ **Example 2** Find the spherical coordinates of the point that has rectangular coordinates

$$(x, y, z) = (4, -4, 4\sqrt{6})$$

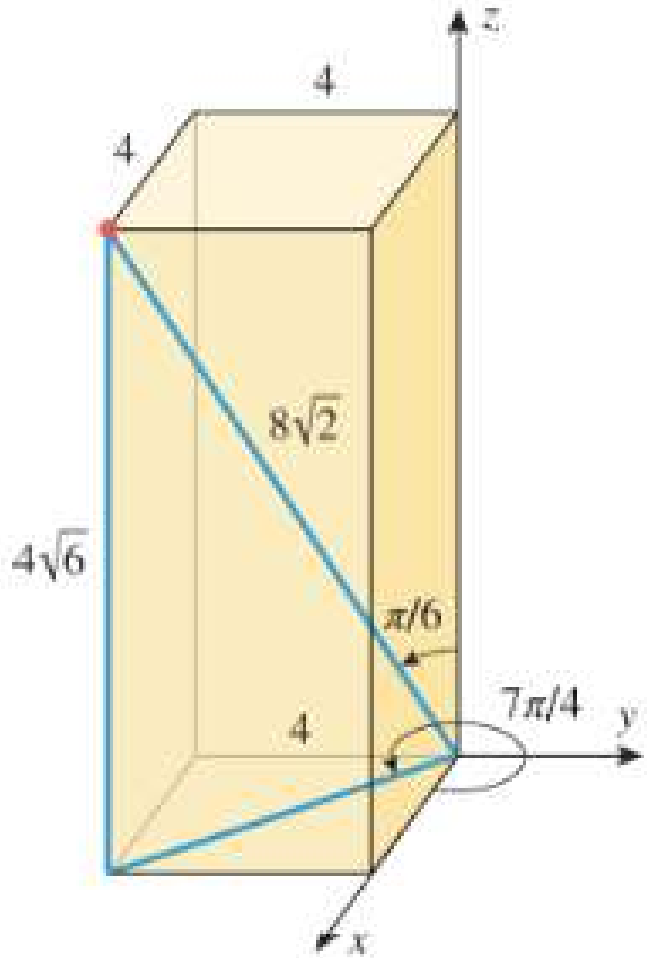
$$\rho = \sqrt{x^2 + y^2 + z^2} = \sqrt{16 + 16 + 96} = \sqrt{128} = 8\sqrt{2}$$

$$\tan\theta = \frac{y}{x} = -1$$

$$\cos\phi = \frac{z}{\sqrt{x^2 + y^2 + z^2}} = \frac{4\sqrt{6}}{8\sqrt{2}} = \frac{\sqrt{3}}{2}$$

From the restriction $0 \le \theta < 2\pi$ and the computed value of $\tan\theta$, the possibilities for θ are $\theta = 3\pi/4$ and $\theta = 7\pi/4$. However, the given point has a negative y-coordinate, so we must have $\theta = 7\pi/4$. Moreover, from the restriction $0 \le \phi \le \pi$ and the computed value of $\cos\phi$, the only possibility for ϕ is $\phi = \pi/6$. Thus, the spherical coordinates of the point

$$\text{are } (\rho, \theta, \phi) = (8\sqrt{2}, 7\pi/4, \pi/6)$$



Exercise

1–2 Convert from rectangular to cylindrical coordinates.

1. (a) $(4\sqrt{3}, 4, -4)$ (b) $(-5, 5, 6)$
(c) $(0, 2, 0)$ (d) $(4, -4\sqrt{3}, 6)$

2. (a) $(\sqrt{2}, -\sqrt{2}, 1)$ (b) $(0, 1, 1)$
(c) $(-4, 4, -7)$ (d) $(2, -2, -2)$

3–4 Convert from cylindrical to rectangular coordinates.

3. (a) $(4, \pi/6, 3)$ (b) $(8, 3\pi/4, -2)$
(c) $(5, 0, 4)$ (d) $(7, \pi, -9)$

4. (a) $(6, 5\pi/3, 7)$ (b) $(1, \pi/2, 0)$
(c) $(3, \pi/2, 5)$ (d) $(4, \pi/2, -1)$

5–6 Convert from rectangular to spherical coordinates.

5. (a) $(1, \sqrt{3}, -2)$ (b) $(1, -1, \sqrt{2})$
(c) $(0, 3\sqrt{3}, 3)$ (d) $(-5\sqrt{3}, 5, 0)$

6. (a) $(4, 4, 4\sqrt{6})$ (b) $(1, -\sqrt{3}, -2)$
(c) $(2, 0, 0)$ (d) $(\sqrt{3}, 1, 2\sqrt{3})$

7–8 Convert from spherical to rectangular coordinates.

7. (a) $(5, \pi/6, \pi/4)$ (b) $(7, 0, \pi/2)$
(c) $(1, \pi, 0)$ (d) $(2, 3\pi/2, \pi/2)$

8. (a) $(1, 2\pi/3, 3\pi/4)$ (b) $(3, 7\pi/4, 5\pi/6)$
(c) $(8, \pi/6, \pi/4)$ (d) $(4, \pi/2, \pi/3)$

Python program

Python program that provides a menu for the user to choose a coordinate transformation. It includes conversions between Cartesian, Cylindrical, and Spherical coordinates in both directions.

```
import math

def cartesian_to_cylindrical(x, y, z):

    r = math.sqrt(x**2 + y**2)

    theta = math.atan2(y, x)  # Angle in radians

    return r, theta, z

def cylindrical_to_cartesian(r, theta, z):

    x = r * math.cos(theta)

    y = r * math.sin(theta)

    return x, y, z

def cartesian_to_spherical(x, y, z):

    r = math.sqrt(x**2 + y**2 + z**2)

    theta = math.atan2(y, x)  # Azimuthal angle in radians

    phi = math.acos(z / r) if r != 0 else 0  # Polar angle in radians

    return r, theta, phi

def spherical_to_cartesian(r, theta, phi):

    x = r * math.sin(phi) * math.cos(theta)
```

```
    y = r * math.sin(phi) * math.sin(theta)
    z = r * math.cos(phi)
    return x, y, z
def cylindrical_to_spherical(r, theta, z):
    rho = math.sqrt(r**2 + z**2)
    phi = math.atan2(r, z)  # Polar angle in radians
    return rho, theta, phi
def spherical_to_cylindrical(rho, theta, phi):
    r = rho * math.sin(phi)
    z = rho * math.cos(phi)
    return r, theta, z
# Menu for user input
while True:
    print("\nCoordinate Transformation Options:")
    print("1. Cartesian to Cylindrical")
    print("2. Cylindrical to Cartesian")
    print("3. Cartesian to Spherical")
    print("4. Spherical to Cartesian")
    print("5. Cylindrical to Spherical")
    print("6. Spherical to Cylindrical")
    print("0. Exit")
   choice = input("Enter your choice (0-6): ")
```

```
if choice == '0':
    print("Exiting the program.")
    break
if choice in ['1', '2', '3', '4', '5', '6']:
    if choice in ['1', '3']:
        x = float(input("Enter x: "))
        y = float(input("Enter y: "))
        z = float(input("Enter z: "))
    elif choice == '2':
        r = float(input("Enter r: "))
        theta = float(input("Enter theta (in radians): "))
        z = float(input("Enter z: "))
    elif choice == '4':
        r = float(input("Enter r: "))
        theta = float(input("Enter theta (in radians): "))
        phi = float(input("Enter phi (in radians): "))
    elif choice == '5':
        r = float(input("Enter r: "))
        theta = float(input("Enter theta (in radians): "))
        z = float(input("Enter z: "))
    elif choice == '6':
        rho = float(input("Enter rho: "))
        theta = float(input("Enter theta (in radians): "))
```

```
        phi = float(input("Enter phi (in radians): "))
    # Perform the transformation based on choice
    if choice == '1':
        result = cartesian_to_cylindrical(x, y, z)
        print(f"Cylindrical Coordinates (r, theta, z): {result}")
    elif choice == '2':
        result = cylindrical_to_cartesian(r, theta, z)
        print(f"Cartesian Coordinates (x, y, z): {result}")
    elif choice == '3':
        result = cartesian_to_spherical(x, y, z)
        print(f"Spherical Coordinates (r, theta, phi): {result}")
    elif choice == '4':
        result = spherical_to_cartesian(r, theta, phi)
        print(f"Cartesian Coordinates (x, y, z): {result}")
    elif choice == '5':
        result = cylindrical_to_spherical(r, theta, z)
        print(f"Spherical Coordinates (rho, theta, phi): {result}")
    elif choice == '6':
        result = spherical_to_cylindrical(rho, theta, phi)
        print(f"Cylindrical Coordinates (r, theta, z): {result}")
else:
    print("Invalid choice. Please enter a number between 0 and 6.")
```

Output

```
Coordinate Transformation Options:
1. Cartesian to Cylindrical
2. Cylindrical to Cartesian
3. Cartesian to Spherical
4. Spherical to Cartesian
5. Cylindrical to Spherical
6. Spherical to Cylindrical
0. Exit
Enter your choice (0-6): 2
Enter r: 3
Enter theta (in radians): 1.5
Enter z: 5
Cartesian Coordinates (x, y, z): (0.2122116050031087, 2.9924849598121632, 5.0)

Coordinate Transformation Options:
1. Cartesian to Cylindrical
2. Cylindrical to Cartesian
3. Cartesian to Spherical
4. Spherical to Cartesian
5. Cylindrical to Spherical
6. Spherical to Cylindrical
0. Exit
Enter your choice (0-6): 0
Exiting the program.
```

Applications: Python Implementation (Spherical System)

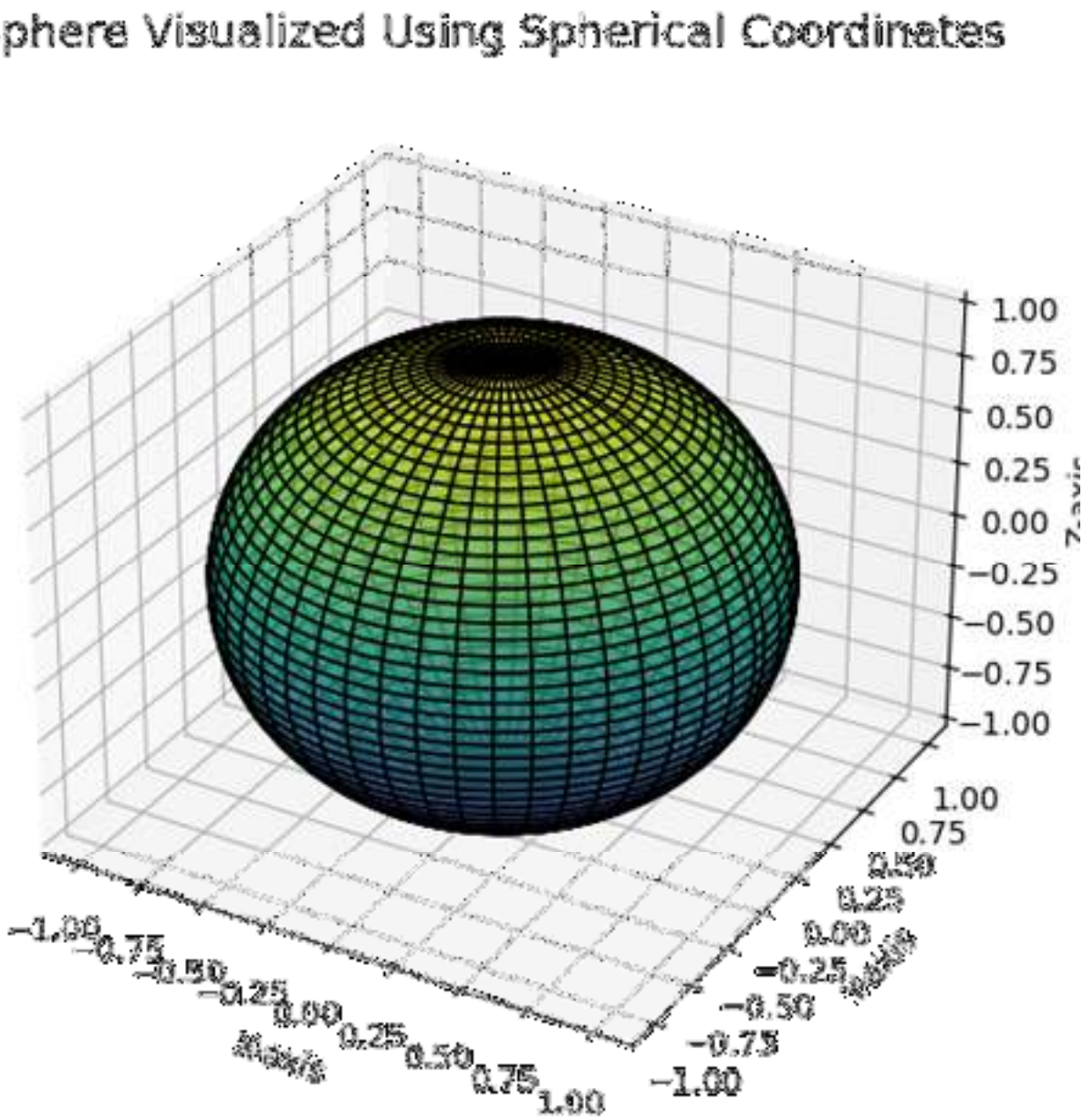
Visualizing a Sphere Using Spherical Coordinates

Code provides a foundational understanding of how spherical coordinates are used to model 3D shapes in computer graphics. It can be extended to more complex applications like texture mapping or animations.

```
import numpy as np

import matplotlib.pyplot as plt

from mpl_toolkits.mplot3d import Axes3D

# Step 1: Define spherical coordinate parameters

phi = np.linspace(0, np.pi, 100)  # Polar angle from 0 to π

theta = np.linspace(0, 2 * np.pi, 100)  # angle from 0 to 2π

phi, theta = np.meshgrid(phi, theta)  # Create a grid of angles

# Step 2: Define radius of the sphere

radius = 1

# Step 3: Convert spherical to Cartesian coordinates

x = radius * np.sin(phi) * np.cos(theta)

y = radius * np.sin(phi) * np.sin(theta)

z = radius * np.cos(phi)

# Step 4: Create a 3D plot

fig = plt.figure(figsize=(8, 6))

ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(x, y, z, cmap='viridis', edgecolor='k', alpha=0.7)

# Step 5: Customize the plot

ax.set_title("Sphere Visualized Using Spherical Coordinates", fontsize=14)

ax.set_xlabel("X-axis")

ax.set_ylabel("Y-axis")

ax.set_zlabel("Z-axis")
```

```
# Step 6: Show the plot

plt.show()
```

Output



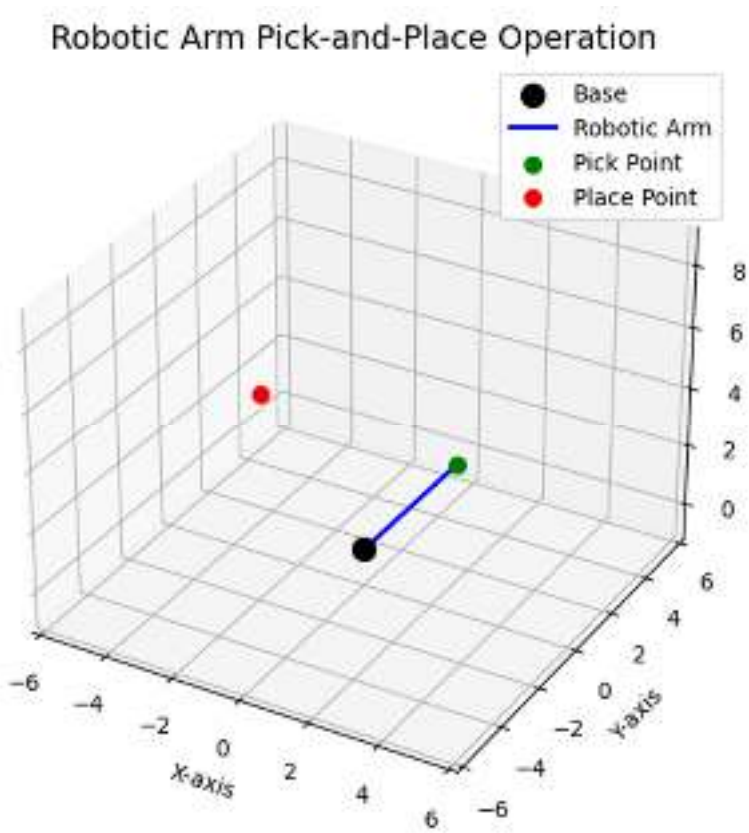
Applications: Python Code Implementation (Cylindrical System)

Robotic Arm Pick-and-Place Using Cylindrical Coordinates

Code illustrates how cylindrical coordinates are essential for simplifying robotic motion in real-world applications.

```
import numpy as np

import matplotlib.pyplot as plt

from mpl_toolkits.mplot3d import Axes3D

# Step 1: Define parameters for the robotic arm

radius = 5  # Fixed radius of the circular motion

height_pick = 0  # Height at the picking point

height_place = 8  # Height at the placing point

theta_pick = np.pi / 2  # Angle where the object is picked (90 degrees)

theta_place = 3 * np.pi / 2  # Angle where the object is placed (270 degrees)

steps = 100  # Number of steps for smooth motion

# Step 2: Generate circular motion for the robotic arm

theta = np.linspace(theta_pick, theta_place, steps)  # Circular motion from pick to place

r = radius * np.ones_like(theta)  # Fixed radius

z = np.linspace(height_pick, height_place, steps)  # Linear vertical motion

# Step 3: Convert cylindrical coordinates to Cartesian coordinates

x = r * np.cos(theta)
```

```
y = r * np.sin(theta)

# Step 4: Plot the robotic arm's motion

fig = plt.figure(figsize=(10, 6))

ax = fig.add_subplot(111, projection='3d')

# Plot the arm's base

ax.scatter(0, 0, 0, color='black', s=100, label="Base")

# Initialize plot elements

arm_line, = ax.plot([], [], [], color='blue', linewidth=2, label="Robotic Arm")

pick_point = ax.scatter([], [], [], color='green', s=50, label="Pick Point")

place_point = ax.scatter([], [], [], color='red', s=50, label="Place Point")

# Mark the pick and place points

ax.scatter(radius * np.cos(theta_pick), radius * np.sin(theta_pick), height_pick, color='green',
s=50)

ax.scatter(radius * np.cos(theta_place), radius * np.sin(theta_place), height_place, color='red',
s=50)

# Customize plot

ax.set_xlim(-radius - 1, radius + 1)

ax.set_ylim(-radius - 1, radius + 1)

ax.set_zlim(-1, height_place + 1)

ax.set_xlabel("X-axis")

ax.set_ylabel("Y-axis")

ax.set_zlabel("Z-axis")

ax.set_title("Robotic Arm Pick-and-Place Operation", fontsize=14)
```

```
ax.legend()

# Step 5: Animate the motion

for i in range(steps):

    # Plot the arm line as a connection between the base and the current point

    arm_line.set_data([0, x[i]], [0, y[i]])

    arm_line.set_3d_properties([0, z[i]])

    plt.pause(0.01)  # Pause to create animation effect

# Final plot display

plt.show()
```

Output